

colesbury Move reflection/replication padding to ATen (#4258) 9495595 · 8 years ago

Name	Last commit message	Last commit date
..		
cmake	Added MAGMA_HOME env var to spe…	8 years ago
conda	Conda packaging (#119)	8 years ago
contrib	[ATen][Scalars] Remove Scalar from ret…	8 years ago
doc	update docs	8 years ago
src	Move reflection/replication padding to A…	8 years ago
tools	Prevent segfaults from undefined aten te…	8 years ago
.flake8	python style fixes	8 years ago
.gitignore	checked cast does it all	8 years ago
.travis.yml	create file so that find_package works in …	8 years ago
CMakeLists.txt	Refactor cudnn code layout / make build …	8 years ago
README.md	Refactor cudnn code layout / make build …	8 years ago

README.md

ATen: A TENsor library

ATen is a simple tensor library thats exposes the Tensor operations in Torch and PyTorch directly in C++11. The wrapper respects the semantics of operators in PyTorch, except minor details due to differences between C++ in Python in the way default arguments are handled. See the documentation for tensors in PyTorch for what these operations do. ATen's API is auto-generated from the same declarations PyTorch uses so the two APIs will track each other over time.

Tensor types are resolved dynamically, such that the API is generic and does not include templates. That is, there is one `Tensor` type. It can hold a CPU or CUDA Tensor, and the tensor may have Doubles, Float, Ints, etc. This design makes it easy to write generic code without templating everything.

See the *generated* `Tensor.h` file and `Functions.h` file for the provided API. Excerpt:

```
Tensor atan2(const Tensor & other) const;
Tensor & atan2_(const Tensor & other);
Tensor pow(Scalar exponent) const;
Tensor pow(const Tensor & exponent) const;
Tensor & pow_(Scalar exponent);
Tensor & pow_(const Tensor & exponent);
Tensor lerp(const Tensor & end, Scalar weight) const;
Tensor & lerp_(const Tensor & end, Scalar weight);
Tensor histc() const;
Tensor histc(int64_t bins) const;
Tensor histc(int64_t bins, Scalar min) const;
Tensor histc(int64_t bins, Scalar min, Scalar max) const;
```

Inplace operations are also provided, and always suffixed by `_` to indicate they will modify the Tensor.

Installation

TH/THC/THNN/THCUNN are provided (as git subtrees), so the repo is standalone. You will need a C++11 compiler, cmake, and the pyyaml python package.

```
# Install pyyaml used by python code generation to read API declaration

# macOS: if you don't have pip
sudo easy_install pip
# Ubuntu: if you don't have pip
apt-get -y install python-pip

# if you don't have pyyaml
sudo pip install pyyaml

mkdir build
```

```
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/where/you/want # specify your dest dir
# cmake .. -DNO_CUDA=true  # for CPU only machines
make install
```

Example usage

Here is a simple example; again, the syntax follows Torch semantics.

```
using namespace at; // assumed in the following

Tensor d = CPU(kFloat).ones({3, 4});
Tensor r = CPU(kFloat).zeros({3,4})
for(auto i = 0; i < 100000; i++) {
  r = r.add(d);
  // equivalently
  r = r + d;
  // or
  r += d;
}
```

Want this running on the GPU?

```
using namespace at; // assumed in the following

Tensor d = CUDA(kFloat).ones({3, 4});
Tensor r = CUDA(kFloat).zeros({3,4})
for(auto i = 0; i < 100000; i++) {
  r = r.add(d);
  // equivalently
  r = r + d;
  // or
  r += d;
}
```

Expressions like `CUDA(kFloat)` are first-class `at::Type` objects that represent the type of a Tensor and are used to create Tensors when their type cannot be inferred. See the *generated* Type header for its API.

See more in sample files.

Creating your kernel

It is easy to create new kernels, thanks to the `dispatch<>()` templated function. Example:

```
// a simple sum kernel (for CPU only)
template<typename T>
struct sum_op {
  // dispatch handles variable arguments for you
  Tensor CPU(const Type & t, Tensor & x_)
```

```
  {
    Tensor x = x_.contiguous();
    auto x_p = x.data<T>();
    int64_t size = x.numel();
    T sum = 0;
    for(int64_t i = 0; i < size; i++) {
      sum += x_p[i];
    }
    return sum;
  };
  Tensor CUDA(Tensor& x) {
    throw std::invalid_argument("device not supported");
  };
};

Tensor a = CPU(kFloat).rand({3, 7});
std::cout << a << std::endl;
std::cout << dispatch<sum_op>(a.type(),a) << " == " << a.sum() << std::
```

Efficient access to tensor elements

When using Tensor-wide operations, the relative cost of dynamic dispatch is very small. However, there are cases, especially in your own kernels, where efficient element-wise access is needed, and the cost of dynamic dispatch inside the element-wise loop is very high. ATen provides *accessors* that are created with a single dynamic check that a Tensor is the type and number of dimensions. Accessors then expose an API for accessing the Tensor elements efficiently:

```
Tensor foo = CPU(kFloat).rand({12,12});

// assert foo is 2-dimensional and holds floats.
auto foo_a = foo.accessor<float,2>();
float trace = 0;

for(int i = 0; i < foo_a.size(0); i++) {
  // use the accessor foo_a to get tensor data.
  trace += foo_a[i][i];
}
```

Accessors are temporary views of a Tensor. They are only valid for the lifetime of the tensor that they view and hence should only be used locally in a function, like iterators.

Using externally created data

If you already have your tensor data allocated in memory (CPU or CUDA), you can view that memory as a Tensor in ATen:

```
float data[] = { 1, 2, 3,
                 4, 5, 6};
```

```
auto f = CPU(kFloat).tensorFromBlob(data, {2,3});
cout << f << endl;
```

These tensors cannot be resized because ATen does not own the memory, but otherwise behave as normal tensors.

Scalars and zero-dimensional tensors

In addition to the `Tensor` objects, ATen also includes `Scalar`s that represent a single number. Like a Tensor, Scalars are dynamically typed and can hold any one of ATen's number types. Scalars can be implicitly constructed from C++ number types. Scalars are needed because some functions like `addmm` take numbers along with Tensors and expect these numbers to be the same dynamic type as the tensor. They are also used in the API to indicate places where a function will *always* return a Scalar value, like `sum`.

```
Tensor addmm(Scalar beta, const Tensor & self,
             Scalar alpha, const Tensor & mat1,
             const Tensor & mat2);
Scalar sum(const Tensor & self);

//usage
Tensor a = ...
Tensor b = ...
Tensor c = ...
Tensor r = addmm(1.0, a, .5, b, c);
```

In addition to Scalars, ATen also allows Tensor objects to be zero-dimensional. These Tensors hold a single value and they can be references to a single element in a larger Tensor. They can be used anywhere a Tensor is expected. They are normally created by operators like `select` which reduce the dimensions of a Tensor.

```
Tensor two = CPU(kFloat).rand({10,20});
two[1][2] = 4;
//~~~~~~~~  zero-dimensional Tensor
```

It is possible to convert between Scalar and zero-dim Tensors:

```
Tensor zero_dim = CPU(kFloat).scalarTensor(4);
Scalar from_tensor = Scalar(zero_dim); //only valid when zero_dim.dim()
```

Avoiding unnecessary CUDA synchronization in your kernels when using Scalars

Moving a single number from the GPU to the CPU introduces a synchronization point that can add latency to your program. In certain cases the result of a GPU operator like `sum` which returns a Scalar may be plugged into another GPU operator as an argument. If

Scalars were always copied to the CPU, this would result in 2 copies. To avoid these synchronizations, Scalar objects can be optionally backed by a zero-dim Tensor, and are only copied to the CPU when requested.

```
auto a = CUDA(kFloat).rand({3,4})
Scalar on_gpu = Scalar(a[1][1]); //backed by zero-dim Tensor
assert(on_gpu.isBackedByTensor());

double value = on_gpu.toDouble(); // copied to CPU, if it was backed by
Scalar svalue = on_gpu.local(); // force the Scalar to become local to

// get the scalar as a zero-dim tensor. If it was already backed
// by a zero-dim Tensor then this op has no synchronization.
// if the Scalar was local on CPU, it performs the copy
Tensor same_tensor = CUDA(kFloat).scalarTensor(on_gpu);
```

Operators aware of the location of Scalars can arrange to do the minimal number of copies required.

Developer notes

ATen relies heavily on code generation to automatically generate headers and implementations for all of the tensor methods it supports. The main entry point for the script which does all this work is `src/ATen/gen.py`, which ingests `src/ATen/Declarations.cwrap`, `src/ATen/nn.yaml`, `src/ATen/native/native_functions.yaml` and the THNN/THCUNN headers and produces all of the headers and wrapping code necessary to generate the ATen interface.

If you need to understand how ATen understands a declaration after all of this processing occurs, it's helpful to look at the generated file `Declarations.yaml` (NB: not cwrap) which contains information for all ATen methods in a uniform manner. This file is utilized by PyTorch which further extends the ATen interface with support for automatic differentation.

Note [ATen preprocessor philosophy]

ATen is designed to be simple to use, and one of the things this implies is that it should not be necessary to use preprocessor macros when using ATen; we would rather provide all symbols, even for functionality that is not available on the system ATen is running on.

This means that internally inside ATen, whereas other libraries might simply omit source files for, e.g., CuDNN, when CuDNN libraries are not installed, ATen will always build these source files, compiling stub functions for anything that is not available. ATen never uses `AT_ENABLED_CUDA()` in header files, and all types in ATen's public API are always available no matter your build configuration.